A particular test can be marked 'pending' using the *pending* and *pendingWith* keywords. The *pendingWith* function takes a string message as an argument, as illustrated below:

```
import Test.Hspec

main = hspec $ do
  describe "Testing equality" $ do
    it "returns 3 for the sum of 2 and 1" $ do
      3 `shouldBe` (2 + 1)

  describe "Testing subtraction" $ do
    it "returns 3 when subtracting 1 from 4" $ do
      pendingWith "need to add test"
```

The corresponding output for the above code snippet is provided below:

```
ghci> main

Testing equality
  returns 3 for the sum of 2 and 1
Testing subtraction
  returns 3 when subtracting 1 from 4
    # PENDING: need to add test

Finished in 0.0006 seconds
2 examples, 0 failures, 1 pending
```

You can use the *after* and *before* keywords to specify setup and tear down functions before running a test. For example:

```
import Test.Hspec

getInt :: IO Int
getInt = do
  putStrLn "Enter number:"
  number <- readLn
  return number

afterPrint :: ActionWith Int
afterPrint 3 = print 3

main = hspec $ before getInt $ after afterPrint $ do
  describe "should be 3" $ do
    it "should successfully return 3" $ \n -> do
      n `shouldBe` 3
```

Executing the above code yields the following output:

```
ghci> main

should be 3
Enter number:
3
3
  should successfully return 3

Finished in 0.6330 seconds
1 example, 0 failures
```

The different options used with Hspec can be listed with the *--help* option, as follows:

```
$ runhaskell file.hs --help
Usage: spec.hs [OPTION]...

OPTIONS
                --help              display this help and exit
  -m PATTERN    --match=PATTERN     only run examples that match
                                    given PATTERN
                --color             colorize the output
                --no-color          do not colorize the output
 -f FORMATTER  --format=FORMATTER   use a custom formatter;
                                    this can be one of:
                                      specdoc
                                      progress
                                      failed-examples
                                      silent
  -o FILE  --out=FILE    write output to a file instead of STDOUT
           --depth=N     maximum depth of generated test values for
                         SmallCheck properties
  -a N     --qc-max-success=N  maximum number of successful tests
                               before a QuickCheck property succeeds
           --qc-max-size=N     size to use for the biggest
                               test cases
           --qc-max-discard=N  maximum number of discarded
                               tests per
                               successful test before giving up
            --seed=N           used seed for QuickCheck properties
            --print-cpu-time   include used CPU time in summary
            --dry-run          pretend that everything passed;
                               don't verify anything
            --fail-fas         abort on first failure
  -r       --rerun      rerun all examples that failed in the
                        previously test run (only works in GHCi)
```

Other than the *shouldBe* expectation, you can also use assertions like *shouldReturn*, *shouldSatisfy*, and *shouldThrow*. Examples of each are given below:

```
import Test.Hspec
import Control.Exception (evaluate)

main = hspec $ do
  describe "shouldReturn" $ do
    it "should successfully return 3" $ do
      return 3 `shouldReturn` 3
```